

第7章: CRUD実装（一覧表示・新規登録）

いよいよアプリの中心機能を作ります。この章では CRUD のうち R（Read＝一覧表示）と C（Create＝新規登録）を実装します。第6章で用意した `lib/books.ts` の関数と、第4章で学んだ `FlatList`・`TextInput` を使います。コードは全行コメント付きで、初めての書き方はすべて解説します。

1. この章のゴール

- Supabaseから本のデータを取得し、`FlatList` で一覧表示する
- 「読込中」の表示や、本が0件のときの表示も用意する
- 入力フォームで新しい本を登録し、保存後に一覧へ戻る

2. 書籍一覧画面を作る（Read）

第6章で仮置きした `app/index.tsx` を、本物の一覧画面に作り替えます。少しずつ理解できるよう、まず完成形を示し、その後ポイントを解説します。

▼ このコードがやること（先に日本語で）：Supabase（データの保管庫）から本のデータを取り出し、`FlatList`（たくさんの項目を効率よく縦に並べて表示する部品）で一覧画面を作ります。読み込んでいる間は「ぐるぐる」（`ActivityIndicator`）を出し、本が0件なら案内文を出す、という気配りも入れます。カギになるのは `useFocusEffect`（この画面が表示されるたびに実行されるしくみ）で、これのおかげで登録画面から戻った瞬間に最新の本が並びます。1行ずつの細かい意味はコード内のコメントで説明しているので、まずは「データを取って一覧に並べる画面」という全体像をつかんでください。

```

// app/index.tsx - 書籍一覧画面 (Read)

import { useState, useCallback } from "react";
import { View, Text, FlatList, Pressable, StyleSheet, ActivityIndicator } from "react-native";
import { useRouter, useFocusEffect } from "expo-router";
import { fetchBooks } from "../lib/books"; // 第6章で作った「全件取得」関数
import { Book } from "../types/book"; // 第6章で作ったBook型

export default function HomeScreen() {
  const router = useRouter(); // 画面移動の道具 (第4章)
  const [books, setBooks] = useState<Book[]>([]); // 本のリストをstateで持つ。<Book[]>で型を指定。初期値は空配列
  const [loading, setLoading] = useState(true); // 読込中かどうか。最初はtrue (読込中)

  // データを読み込む関数。async (待つ処理を含む)
  const load = useCallback(async () => {
    try { // try : この中の処理を試す。エラーが出たらcatchへ飛ぶ
      setLoading(true); // 読込開始 : ローディング表示をオンに
      const data = await fetchBooks(); // Supabaseから全件取得 (結果が返るまで待つ)
      setBooks(data); // 取得したデータをstateにセット → 画面が更新される
    } catch (e) { // catch : tryの中でエラーが起きたらここに来る。eにエラーが入る
      console.log("読み込みエラー:", e); // エラー内容を表示 (学習用。本番は画面に出すと親切)
    } finally { // finally : 成功・失敗どちらでも最後に必ず実行
      setLoading(false); // 読込終了 : ローディング表示をオフに
    }
  }, []);

  // useFocusEffect : 「画面が表示される (フォーカスされる) たび」に実行するExpo Routerのフック
  // 登録画面から戻ってきたときも再取得され、追加した本がすぐ反映される
  useFocusEffect(
    useCallback(() => {
      load();
    }, [load])
  );

  // 読込中はぐるぐる回るインジケータを表示して、ここで描画を終える (早期return)
  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#1e40af" /> {/* ActivityIndicator : 読込中のぐるぐる */}
      </View>
    );
  }
}

```

```

return (
  <View style={styles.container}>
    <FlatList
      data={books} // 表示するデータ（本の配列）
      keyExtractor={({item}) => item.id} // 各本を区別するキー（idを使う）
      contentContainerStyle={{ padding: 16, gap: 10 }} // リスト内側の余白と要素間の隙間
      // ListEmptyComponent : dataが空のときに表示する内容
      ListEmptyComponent={
        <Text style={styles.empty}>まだ本が登録されていません。右下の+から追加しましょう。
      </Text>
    }
    // renderItem : 1冊分の見た目。itemにその本が入る（第4章のFlatList参照）
    renderItem={({ item }) => (
      <View style={styles.card}>
        <Text style={styles.title}>{item.title}</Text>
        <Text style={styles.author}>著者: {item.author}</Text>
        {/* ステータスを色付きバッジで表示。statusの値で色を変える（下のgetStatusStyle） */}

        <View style={[styles.badge, getStatusStyle(item.status)]>
          <Text style={styles.badgeText}>{item.status}</Text>
        </View>
      </View>
    )}
  </View>

  {/* 画面右下に浮かぶ追加ボタン（FAB = Floating Action Button） */}
  <Pressable style={styles.fab} onPress={() => router.push("/new")}>
    <Text style={styles.fabText}>+</Text>
  </Pressable>
</View>
);
}

// ステータスごとに色を変える補助関数。引数statusに応じて背景色を返す
function getStatusStyle(status: string) {
  if (status === "読了") return { backgroundColor: "#dcfce7" }; // 緑系
  if (status === "読書中") return { backgroundColor: "#d1ecf1" }; // 青系
  return { backgroundColor: "#fff3cd" }; // それ以外（未読）は黄
系
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8f9fa" },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  empty: { textAlign: "center", color: "#999", marginTop: 40, paddingHorizontal: 20 },
  card: {
    backgroundColor: "white",
    borderRadius: 10,
    padding: 14,
  },
});

```

```

borderWidth: 1,
borderColor: "#e2e8f0",
},
title: { fontSize: 15, fontWeight: "bold", color: "#1e293b" },
author: { fontSize: 12, color: "#64748b", marginTop: 3 },
badge: { alignSelf: "flex-start", marginTop: 8, paddingVertical: 3,
paddingHorizontal: 10, borderRadius: 20 },
badgeText: { fontSize: 11, fontWeight: "bold", color: "#334155" },
fab: {
  position: "absolute", // position:"absolute" : 通常の流れから外して自由配置
  right: 20, bottom: 30, // 右から20、下から30の位置に固定
  width: 56, height: 56, borderRadius: 28, // 直径56の円 (borderRadiusを半分にすると円に
なる)
  backgroundColor: "#1e40af",
  justifyContent: "center", alignItems: "center",
  shadowColor: "#000", shadowOpacity: 0.3, shadowRadius: 4, shadowOffset: { width: 0,
height: 2 },
  elevation: 5, // elevation : Androidの影の深さ (iOSはshadow系で指定)
},
fabText: { color: "fff", fontSize: 28, lineHeight: 30 },
});

```

2.1 重要ポイントの解説

- ① `useState<Book[]>([])` の `<Book[]>` `useState` に `<Book[]>` (山カッコで型を指定) を付けて「このstateはBookの配列だ」と明示しています。これは第2章で学んだジェネリクス (入れ物の型に"中身の型"を渡すしくみ) です。 `useState` という入れ物に「中身は `Book[]` (本の配列) ですよ」と教えているわけです。こうすると `books` に対して `item.title` などを書くとき、TypeScriptが補完・チェックしてくれます。初期値の `[]` (空配列) は「最初は本が0冊」という意味です。
- ② `try / catch / finally` サーバー通信は失敗することがあります (電波が悪いなど)。 `try` で試し、失敗したら `catch` でエラーを受け止め、 `finally` で「成功でも失敗でも必ずやること (ローディングを消す)」を書きます。これでアプリが固まりません。

try / catch / finally の流れ: - `try { ... } ...` まずここを実行してみる。 - `catch (e) { ... } ...` `try` の中でエラーが起きたら、ここに飛んでくる (`e` にエラー情報)。 - `finally { ... } ...` 成功・失敗どちらでも、最後に必ず実行される。

- ③ `useFocusEffect` を使う理由 第3章で `useEffect` (最初の1回だけ実行) を学びましたが、ここでは `useFocusEffect` (画面が表示されるたびに実行) を使います。理由は「新規登録画面で本を追加して一覧に戻ったとき、戻った瞬間に再取得して最新のリストを見せたい」からです。 `useEffect` だと最初の1回しか動かず、追加した本がすぐ反映されません。

useCallback とは？ なぜ **useFocusEffect** と組み合わせるの？ 「おまじない」で済ませず、理由を説明します。

まず前提として、Reactのコンポーネント（画面の関数）は、stateが変わるたびに関数ぜんたいが何度も再実行されます。すると、その中で定義している **load** のような関数は、**実行のたびに"別物"**として作り直されてしまいます（中身は同じでも、コンピュータから見ると毎回新しい関数）。

ここで困ったことが起きます。**useFocusEffect** は「渡された関数が"別物"に変わったら、処理をやり直す」性質があります。もし **load** が毎回作り直されると、**useFocusEffect** は「関数が変わった！」と勘違いし、**何度も無駄に再取得（最悪は無限ループ）**してしまうのです。

これを防ぐのが **useCallback**（ユーズ・コールバック）です。「関数を**毎回作り直さず、覚えておく（メモする＝メモ化）**」フックで、第2引数の依存配列（**[load]**の部分）の中身が変わらない限り、**同じ関数を使い回します**。これで **useFocusEffect** が「変わっていない」と正しく認識でき、無駄な再実行が止まります。

まとめると **useFocusEffect(useCallback(() => { load() }, [load]))** は「**画面が表示されるたびに load を呼ぶ。ただし load は毎回作り直さず使い回す**」という意味です。**useFocusEffect** と **useCallback** がセットで出てくるのは、この「無駄な再実行を防ぐ」ためのお決まりだと理解しておけば十分です。

- ④ **ListEmptyComponent** **FlatList** に標準で備わる便利機能で、「データが0件のときに表示する内容」を指定できます。「まだ本がありません」という案内を出して、空っぽの寂しい画面を防ぎます。

3. 新規登録フォームを作る（Create）

第6章で仮置きした **app/new.tsx** を、本物の入力フォームに作り替えます。

▼ このコードがやること（先に日本語で）：新しい本を登録するための入力フォームを作ります。タイトル・著者・ステータス・メモの各入力欄を **TextInput** で用意し、入力された文字を **useState**（画面が覚えておく値）で1つずつ管理します。保存ボタンを押すと、まず「タイトルと著者が空でないか」をチェック（バリデーション）し、問題なければ **createBook**（第6章で作った追加関数）でデータを保存してから一覧画面へ戻ります。保存中はボタンを押せなくて二重登録を防ぐ工夫も入っています。各行の細かい意味はコード内のコメントにあるので、ここでは「入力 → チェック → 保存 → 戻る」という流れをつかんでおけば十分です。

```
// app/new.tsx - 新規登録画面 (Create)

import { useState } from "react";
import { View, Text, TextInput, Pressable, StyleSheet, ScrollView, Alert } from "react-native";
import { useRouter } from "expo-router";
import { createBook } from "../lib/books"; // 第6章で作った「追加」関数

// ステータスの選択肢を配列で用意 (ボタンを並べるのに使う)
const STATUS_OPTIONS = ["未読", "読書中", "読了"];

export default function NewBookScreen() {
  const router = useRouter();

  // 入力欄ごとにstateを用意する (第4章の制御コンポーネント)
  const [title, setTitle] = useState(""); // タイトル (最初は空文字)
  const [author, setAuthor] = useState(""); // 著者
  const [status, setStatus] = useState("未読"); // ステータス (初期値は未読)
  const [memo, setMemo] = useState(""); // メモ
  const [saving, setSaving] = useState(false); // 保存処理中かどうか (連打防止に使う)

  // 保存ボタンを押したときの処理
  const handleSave = async () => {
    // 入力チェック (バリデーション): タイトルと著者は必須
    // .trim(): 文字列の前後の空白を除く。空白だけの入力を「未入力」とみなすため
    if (title.trim() === "" || author.trim() === "") {
      Alert.alert("入力エラー", "タイトルと著者は必須です"); // Alert.alert(タイトル, 本文) : 警告ダイアログを出す
      return; // 処理を中断 (保存しない)
    }

    try {
      setSaving(true); // 保存開始 (ボタンを無効化して二重送信を防ぐ)
      // createBook(...): 第6章の追加関数。入力値をオブジェクトにまとめて渡す
      await createBook({
        title: title.trim(), // 前後空白を除いて保存
        author: author.trim(),
        status: status,
        memo: memo.trim() === "" ? null : memo.trim(), // メモが空ならnull、あれば値を保存 (三項演算子)
      });
      // 成功したら一覧画面へ戻る。一覧側のuseFocusEffectが再取得し、追加した本が表示される
      router.back();
    } catch (e) {
      Alert.alert("保存に失敗しました", "通信状況を確認してもう一度お試しください");
      console.log("保存エラー:", e);
    } finally {
      setSaving(false); // 成功・失敗どちらでも保存状態を解除
    }
  }
}
```

```

};

return (
  // ScrollView : 入力欄が多くキーボードで隠れるのを防ぐため、スクロール可能にする
  <ScrollView style={styles.container} contentContainerStyle={{ padding: 20, gap: 18
}}>
    {/* タイトル入力 */}
    <View>
      <Text style={styles.label}>タイトル *</Text>
      <TextInput
        style={styles.input}
        placeholder="書籍のタイトルを入力"
        value={title} // stateと結びつける
        onChangeText={setTitle} // 入力が変わるたびsetTitleで更新
(setTitle(text)の短縮形)
      />
    </View>

    {/* 著者入力 */}
    <View>
      <Text style={styles.label}>著者 *</Text>
      <TextInput
        style={styles.input}
        placeholder="著者名を入力"
        value={author}
        onChangeText={setAuthor}
      />
    </View>

    {/* ステータス選択 (3つのボタンから1つ選ぶ) */}
    <View>
      <Text style={styles.label}>ステータス</Text>
      <View style={styles.statusRow}>
        {/* STATUS_OPTIONS配列をmapでボタンに変換 (第2章のmap) */}
        {STATUS_OPTIONS.map((option) => (
          <Pressable
            key={option} // map で並べる要素には必ずkeyが必要
            // style に配列を渡すと複数スタイルを合成できる。選択中なら強調スタイルを追加
            style={[styles.statusButton, status === option &&
styles.statusButtonActive]}
            onPress={() => setStatus(option)} // 押したらそのステータスを選択
          >
            <Text style={[styles.statusText, status === option &&
styles.statusTextActive]}>
              {option}
            </Text>
          </Pressable>
        ))}
      </View>
    </View>
  </View>
);

```



```

    { /* メモ入力（複数行） */ }
    <View>
      <Text style={styles.label}>メモ</Text>
      <TextInput
        style={[styles.input, styles.textarea]}
        placeholder="感想や覚え書きなど"
        value={memo}
        onChangeText={setMemo}
        multiline // multiline : 複数行入力を許可する
        numberOfLines={4} // 表示上の高さの目安（4行分）
      />
    </View>

    { /* 保存ボタン。saving中は無効化&文言変更 */ }
    <Pressable
      style={[styles.saveButton, saving && styles.saveButtonDisabled]}
      onPress={handleSave}
      disabled={saving} // disabled : trueの間は押せなくする
    >
      <Text style={styles.saveButtonText}>{saving ? "保存中..." : "保存する"}</Text>
    </Pressable>
  </ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  label: { fontSize: 13, fontWeight: "600", color: "#334155", marginBottom: 6 },
  input: {
    backgroundColor: "#fff",
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8,
    paddingHorizontal: 12, paddingVertical: 10, fontSize: 14,
  },
  textarea: { height: 100, textAlignVertical: "top" }, // textAlignVertical:"top" : 複数行で上揃えにする
  statusRow: { flexDirection: "row", gap: 8 }, // ステータスボタンを横並びに
  statusButton: {
    flex: 1, alignItems: "center", paddingVertical: 10,
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8, backgroundColor: "#fff",
  },
  statusButtonActive: { borderColor: "#1e40af", backgroundColor: "#eff6ff" }, // 選択中の強調
  statusText: { fontSize: 13, color: "#475569" },
  statusTextActive: { color: "#1e40af", fontWeight: "700" },
  saveButton: {
    backgroundColor: "#1e40af", paddingVertical: 14, borderRadius: 10, alignItems:
    "center", marginTop: 8,
  },
});

```



```
saveButtonDisabled: { backgroundColor: "#94a3b8" }, // 保存中はグレーにして押せない雰囲気  
saveButtonText: { color: "#fff", fontWeight: "bold", fontSize: 15 },  
});
```

3.1 重要ポイントの解説

① 入力チェック（バリデーション） `title.trim() === ""` で「タイトルが空（または空白だけ）」かを判定し、必須項目が未入力なら `Alert.alert` で警告を出して保存を中断します。`trim()` は前後の空白を取り除くので、「スペースだけ入力」も未入力扱いにできます。

Alert.alert とは？ スマホOS標準の「ポップアップ警告」を出す関数です。`Alert.alert("タイトル", "本文")` の形で、ユーザーに「OK」を押させるダイアログを表示します。入力ミスや確認メッセージに使います。

② 二重送信の防止（`saving state`） 保存ボタンを連打されると、同じ本が何度も登録されてしまいます。`saving` というstateで「保存処理中」を管理し、その間はボタンを `disabled` （押せない）にし、文言を「保存中...」に変えます。これはアプリ品質を上げる定番テクニックです。

③ `style` に配列を渡す `style={[styles.statusButton, status === option && styles.statusButtonActive]}` のように `style` に配列を渡すと、複数のスタイルを重ねられます。`条件 && スタイル` は「条件がtrueのときだけそのスタイルを適用」という意味で、選択中のボタンだけを強調するのに使っています。

条件 && 値 の意味: `&&` （アンド）を使った `status === option && styles.statusButtonActive` は、「左がtrueなら右の値、falseなら何もしない（false）」という短い書き方です。Reactでは「条件を満たすときだけ何かを表示／適用する」のに多用します。

④ `onChangeText={setTitle}` の短縮 第4章では `onChangeText={({text}) => setTitle(text)}` と書きましたが、これは `onChangeText={setTitle}` と短く書けます。「受け取った文字をそのまま `setTitle` に渡す」だけなので、関数を直接指定できるのです。

4. 動作確認

`npx expo start` でアプリを起動し、次を確認します。

1. 一覧画面に、第5章で入れたテストデータ（3冊）が表示される。
2. 右下の「+」を押すと登録画面へ遷移する。
3. タイトル・著者を空のまま保存 → 「入力エラー」の警告が出る。
4. タイトル・著者・ステータスを入力して「保存する」 → 一覧画面に戻り、追加した本が一番上に表示される（`created_at` の降順で並べているため）。

5. Supabaseのダッシュボード（Table Editor）でも、追加した行が増えているのを確認できる。

よくあるつまずき: - 一覧が空のまま → 第5章のRLSポリシー設定を確認（未設定だとデータが読めない）。 - 追加しても一覧に出ない → `index.tsx` が `useEffect` でなく `useFocusEffect` になっているか確認。 - 保存でエラー → `lib/books.ts` の `createBook` と、`.env` の接続情報を確認。

5. この章のまとめ

- Read: `fetchBooks()` で全件取得し、`FlatList` で一覧表示。`ActivityIndicator` で読込中、`ListEmptyComponent` で0件時を表示
- `useFocusEffect` を使い、登録画面から戻った瞬間に最新データを再取得
- `try / catch / finally` で通信エラーに備える
- Create: `TextInput` の入力をstateで管理し、`createBook()` で登録。バリデーションと二重送信防止も実装
- `Alert.alert` でユーザーへの通知、`style` 配列 + `&&` で条件付きスタイル

次の章へ: 本を「表示」「追加」できるようになりました。第8章では残りの U（Update＝編集）と D（Delete＝削除）を実装し、CRUDを完成させます。